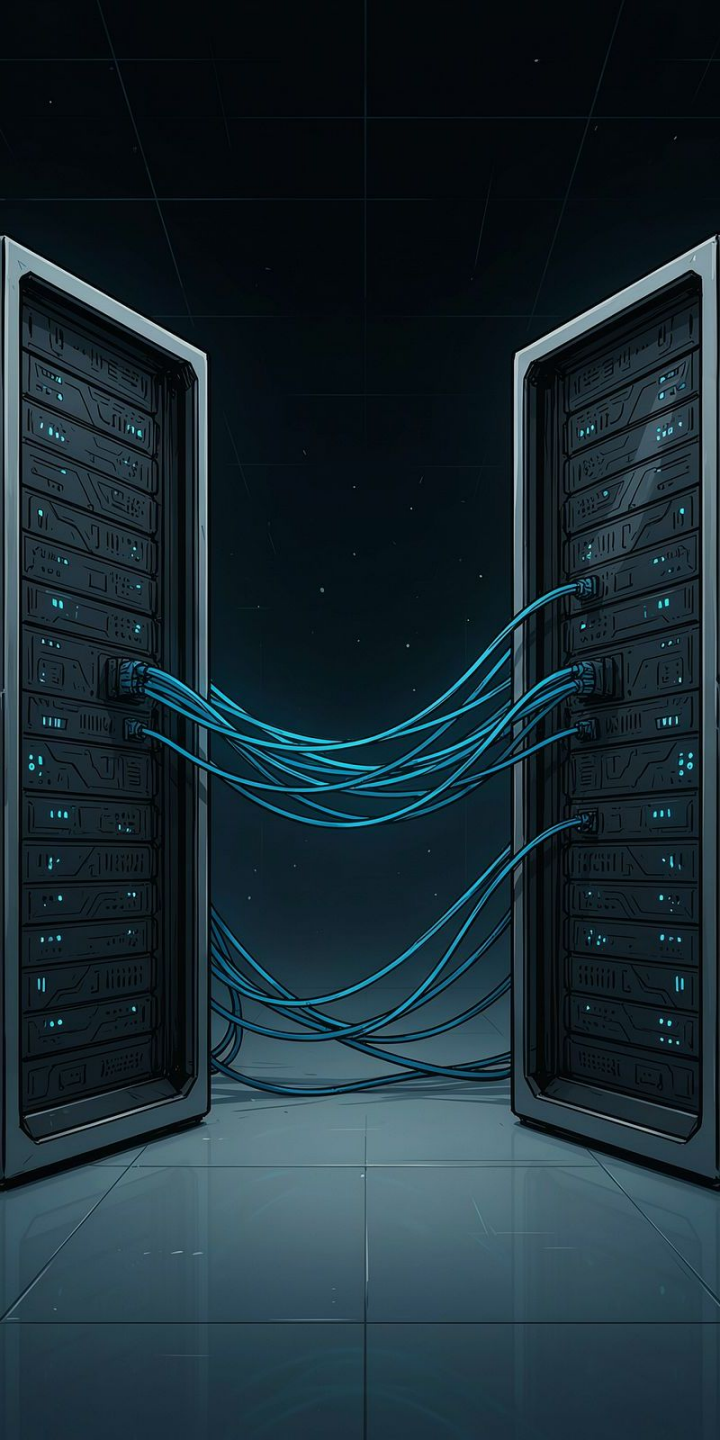




Service Communication in Spring Boot Microservices

How one Spring Boot service talks to another

**Student Service
(8080)**

Initiates REST request

**Service-to-
Service
Call**

**Course Service
(8081)**

Receives and responds

Why Do Services Need to Communicate?

In a microservices architecture, business functionality is intentionally split across multiple independent services. No single service does everything — each owns a specific domain. But to fulfill a real business operation, they must collaborate.

Student Service

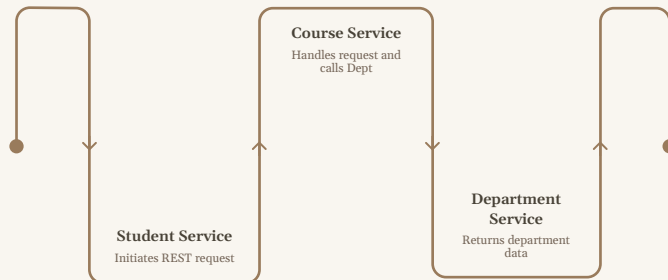
- Student information
- Student registration

Course Service

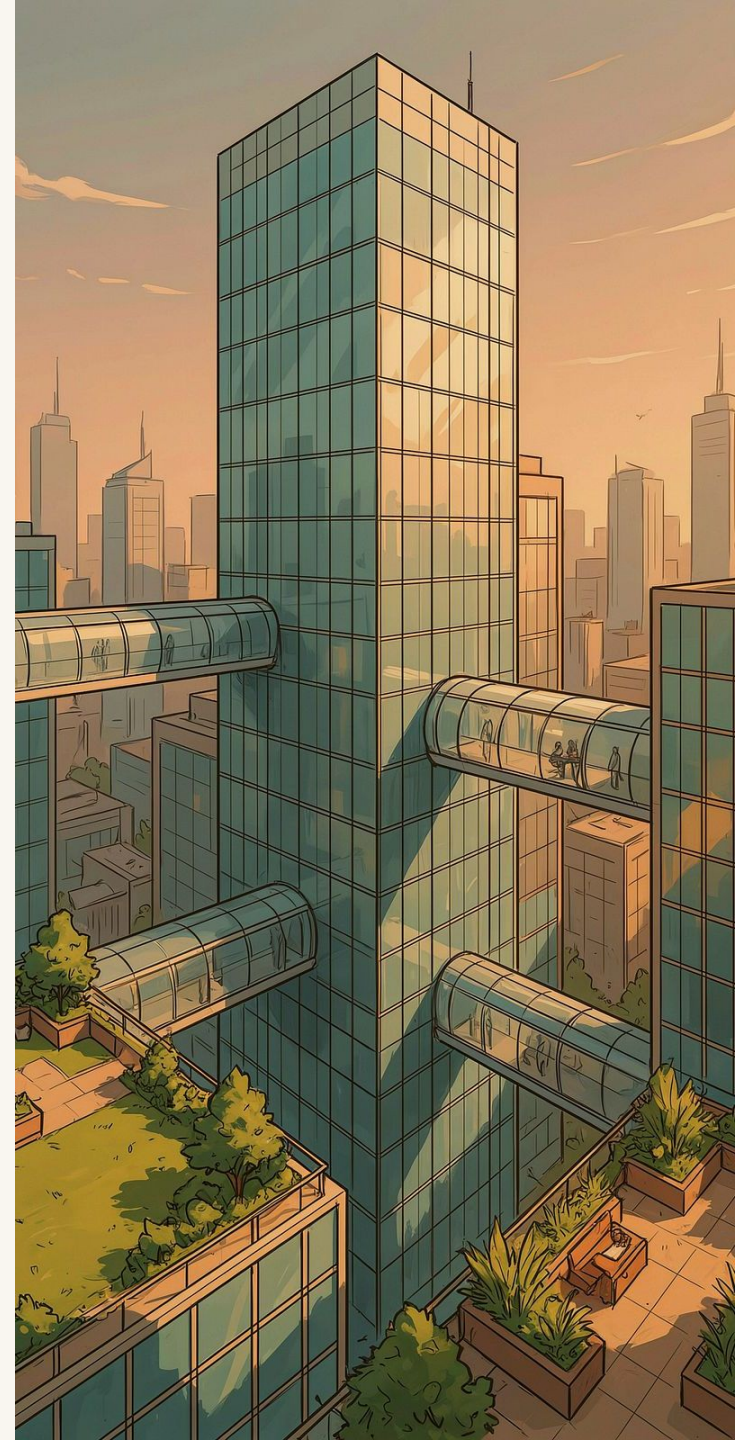
- Course information
- Course enrollment

Department Service

- Department information
- Faculty mapping



- ❏ **Key message:** Microservices are independent, but they still need to work together to complete a full business operation.



Our Running Example

Throughout this session, we use a simple **Student Management** scenario. When a client asks for student details, the Student Service returns its own data — but it does **not** own course information. It must ask the Course Service.

Student Service Response

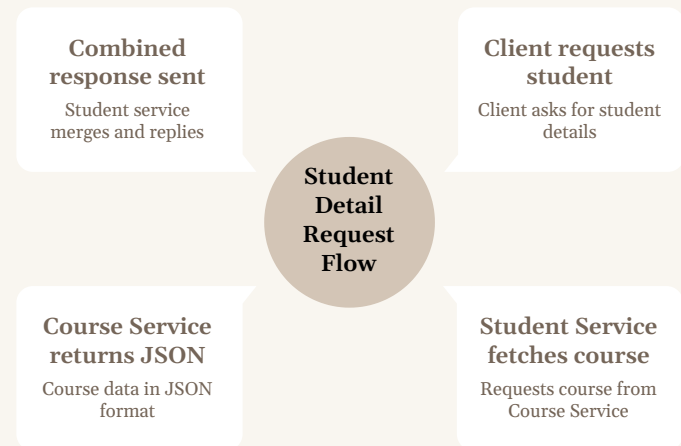
```
{  
  "id": 101,  
  "name": "Rahul",  
  "courseId": 10  
}
```

Student Service owns student data only. It has a `courseId` but no course details.

So it asks Course Service:

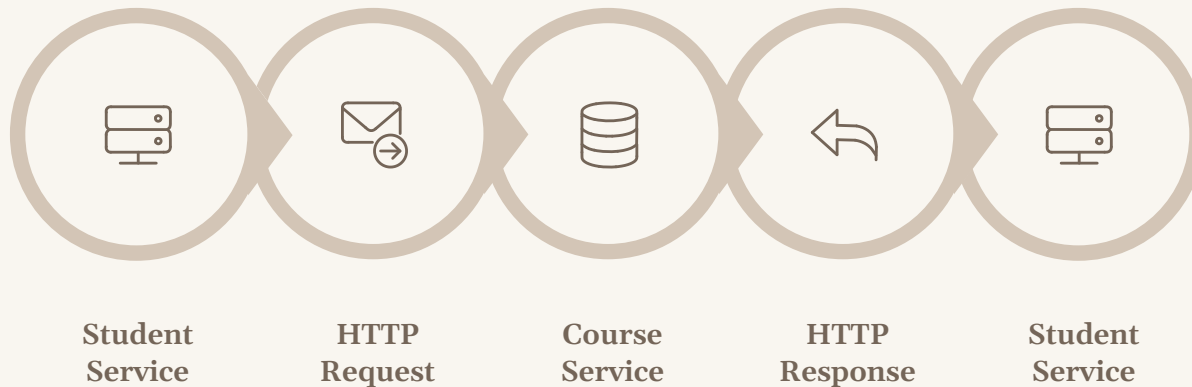
GET /courses/10

Course Service responds with full course details, which Student Service combines into the final response for the client.



What Is Service-to-Service Communication?

Service-to-service communication means one **backend service** sends an HTTP request to another backend service — just like a browser calls an API, except the caller is code, not a human.



HTTP

The transport protocol used between services — the same protocol your browser uses.



REST

Architectural style using HTTP methods (GET, POST, PUT, DELETE) to expose endpoints.



JSON

The data format exchanged between services — lightweight, human-readable, easy to parse.

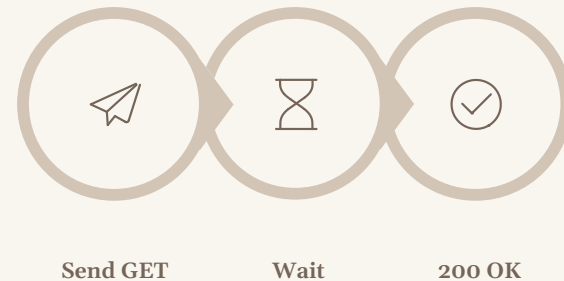


HTTP Status Codes

200 OK, 404 Not Found, 500 Server Error — services communicate success or failure via status codes.

Synchronous Communication

In **synchronous communication**, the calling service sends a request and **waits** — it does nothing else until it receives the response. This is the most straightforward model and is very common in REST-based microservices.



Request

Student Service sends `GET /courses/10` to Course Service.

Wait

Student Service is blocked. It waits for the response before doing anything else.

Response

Course Service replies with `200 OK` and a JSON body. Student Service continues.

HTTP Request and Response — A Real Example

Let's look at exactly what travels over the network when Student Service calls Course Service. This is standard HTTP — nothing magical.

HTTP Request

```
GET /courses/10 HTTP/1.1
Host: course-service:8081
Accept: application/json
```

Student Service sends a standard GET request to Course Service's host and port.

HTTP Response

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "id": 10,
  "name": "Spring Boot",
  "duration": 40
}
```

Course Service responds with a JSON body and a 200 OK status.

- 📌 **Key Principle:** Student Service never touches the Course database directly. It always goes through the Course Service API. This is a fundamental microservices rule — each service owns its own data.

How Spring Boot Calls Another Service

Spring Boot provides **RestClient** (introduced in Spring 6 / Boot 3) as the modern way to make HTTP calls from your Java code. It follows a clean, fluent API style that is easy to read and understand.



```
RestClient restClient = RestClient.create();
```

```
Course course = restClient
```

```
.get()
```

```
.uri("http://localhost:8081/courses/10")
```

```
.retrieve()
```

```
.body(Course.class);
```

`.get()`

Sets the HTTP method to GET.

`.uri()`

Sets the URL of the target service endpoint.

`.retrieve()`

Executes the request and gets the response.

`.body(Course.class)`

Deserializes the JSON response body into a `Course` Java object.

RestClient in Simple Words

Think of RestClient as your Java program's built-in browser. Just as you type a URL in Chrome and get a webpage back, RestClient types a URL in code and gets a JSON response back.

Browser (Human)

```
http://localhost:8081/courses/10
```

You type a URL → Browser sends HTTP GET →
Course Service responds → Browser displays the
result.

RestClient (Java Code)

```
restClient  
    .get()  
    .uri("http://localhost:8081/courses/10")  
    .retrieve()  
    .body(Course.class);
```

Code calls RestClient → RestClient sends HTTP GET →
Course Service responds → Java object is returned.

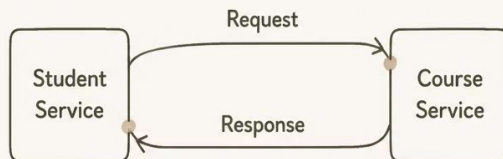
 **Key Message:** RestClient is simply a way for Java code to make HTTP requests — exactly what a browser does, but programmatically.

Synchronous vs. Asynchronous Communication

It's important to understand that synchronous REST is just one way services can communicate. There is another model — asynchronous — where the sender does not wait for an immediate response. We cover the **concept** only; implementation is a future topic.

Synchronous:

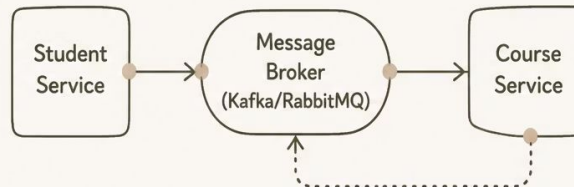
Student Service sends request to Course Service and waits for response



- ✓ direct coupling
- ✓ simple
- ✓ immediate

Asynchronous:

Student Service sends message to Message Broker (Kafka/RabbitMQ), Course Service processes later



- ✓ decoupled
- ✓ eventual consistency

i For this session: We only understand the concept of asynchronous communication. We will **not** implement Kafka or RabbitMQ today. That is an advanced topic for a future session.

What Can Go Wrong?

Service communication over a network introduces a new category of failures that don't exist inside a single application. When Student Service calls Course Service, many things can go wrong.



Connection Refused

Course Service is not running. The TCP connection cannot be established.



Timeout

Course Service is too slow to respond. Student Service waits indefinitely.



500 Server Error

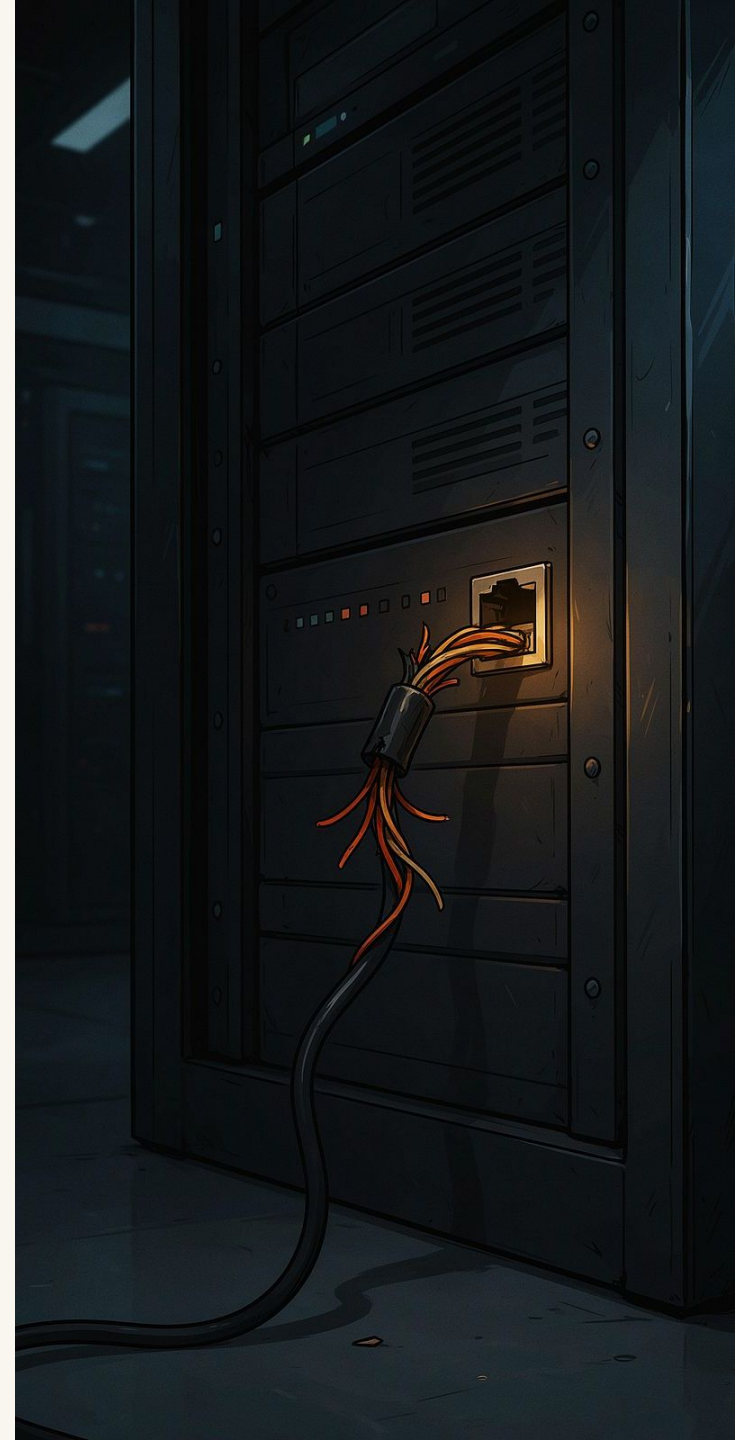
Course Service crashes mid-request and returns an error response.



Service Unavailable

Course Service is overloaded and cannot handle the incoming request.

Remember: Service communication introduces failure *between* services — not just inside them. This is the fundamental challenge of microservices.



The Real Microservices Problem

When one service in a chain fails, the failure can **cascade** upstream. If Course Service is down, Student Service — which depends on it — may also fail or hang, causing the client to see an error.

✓ Happy Path



Client →
Student



Student →
Course



Course →
Database

✗ Course Service Down



Client Call

Student
Service

Course
Service
Down

This introduces the need for resilience strategies. We won't implement them today, but you'll hear these terms soon:

Timeout

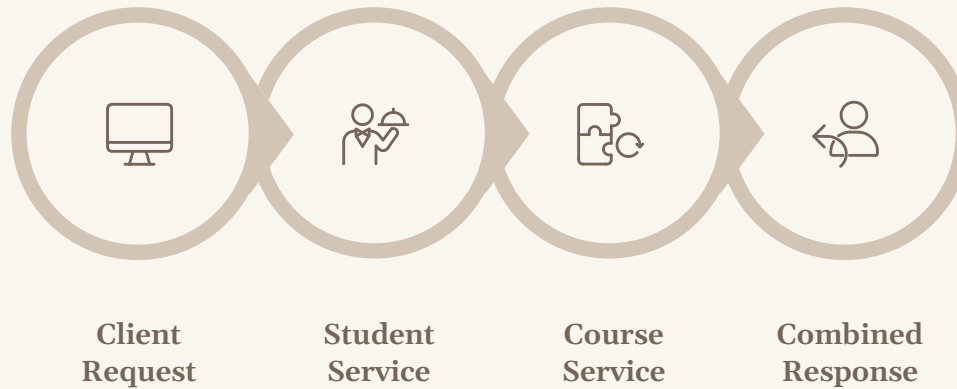
Retry

Circuit Breaker

Fallback

Practical Lab — Architecture

Now let's build it. We'll create two Spring Boot services and wire them together using RestClient. The goal is a real, working inter-service HTTP call you can run and break on your own machine.



Course Service — Port 8081

```
GET /courses/10
```

Response:

```
{
  "id": 10,
  "name": "Spring Boot",
  "duration": 40
}
```

Student Service — Port 8080

```
GET /students/101
```

→ internally calls GET /courses/10

→ returns combined response

Student Service acts as the **orchestrator** — it calls Course Service and merges the data before responding to the client.

Practical Lab — Step-by-Step

Follow these steps in order. After completing Step 7, deliberately break the setup in Step 8 to observe what failure looks like in practice.

01

Create Course Service

New Spring Boot project with `spring-web` dependency, port set to 8081.

02

Expose GET `/courses/{id}`

Return a hardcoded `Course` object as JSON response.

03

Test Course Service Independently

Open `http://localhost:8081/courses/10` in your browser or Postman. Confirm JSON is returned.

04

Create Student Service

New Spring Boot project on port 8080.
Add Student model with `courseId` field.

05

Add RestClient — Call Course Service

Inject `RestClient`, call `http://localhost:8081/courses/{courseId}`, deserialize to `Course`.

06

Return Combined Response

Merge student data + course data into a single response object and return it from `/students/101`.

07

Stop Course Service — Observe Failure

Kill Course Service. Call `/students/101` again. What error does Student Service throw?

Expected Final Result

When everything is working, a single call to Student Service triggers an internal REST call to Course Service, combines both data sets, and returns a unified JSON response to the client.



Client Request

GET http://localhost:8080/students/101



Internal Call by Student Service

GET http://localhost:8081/courses/10



Final Response to Client

```
{
  "studentId": 101,
  "studentName": "Rahul",
  "course": {
    "id": 10,
    "name": "Spring Boot",
    "duration": 40
  }
}
```

Client Request

Student Service 8080

Call Course Service 8081

Merge & Respond

What You Should Remember

Before you close your laptops — here are the five core ideas from today's session. If you understand these, you're ready to build real inter-service communication.

1 Microservices Need to Communicate

Each service owns only its domain. To complete a business operation, services must talk to each other.

2 REST over HTTP Is the Common Mechanism

Services exchange data using HTTP methods, URLs, JSON payloads, and status codes — just like a browser and a server.

3 Synchronous = Request → Wait → Response

The calling service blocks until it gets a reply. Simple, predictable, and the most common pattern for REST APIs.

4 Spring Boot RestClient Makes HTTP Calls Easy

Use `RestClient.create()` with a fluent `.get().uri().retrieve().body()` chain to call any REST endpoint.

5 Downstream Failures Are Real — Plan for Them

If Course Service is down, Student Service can also fail. Communication introduces resilience challenges to tackle in future sessions.

Today we learned how services talk. Next, we learn how to make that communication **reliable**. 🚀